

Trabajo Practico Integrador – Programación 3

DESARROLLO DE FOOD STORE

ALUMNO: HUGO TCACH

Página 2: Índice

Tabla de Contenidos

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	4
3. Dificultades y Soluciones (Análisis de Deuda Técnica)	[Completar número]
4. Enlaces de Entrega (Video y Repositorio)	[Completar número]
5. Bibliografía / Webgrafía	[Completar número]

1. Marco Teórico

El presente proyecto se fundamenta en un ecosistema tecnológico dividido en dos capas principales, diseñadas con una visión arquitectónica orientada a la separación de responsabilidades y la portabilidad:

1.1. Tecnologías de Frontend

- **TypeScript:** Se implementó como lenguaje principal para dotar al código de tipado estático, permitiendo la definición estricta de interfaces y modelos de datos. Esto garantizó la detección temprana de errores en tiempo de desarrollo y una manipulación robusta del DOM nativo.
- **Vite:** Se utilizó como herramienta de construcción y empaquetado (bundler), proporcionando un entorno de desarrollo altamente optimizado y recargas modulares instantáneas (HMR) gracias a su arquitectura basada en ES Modules.
- **HTML5 y CSS3:** Se emplearon para estructurar de forma semántica el contenido y diseñar una interfaz de usuario (UI) limpia y adaptable, garantizando una experiencia de navegación fluida.
- **LocalStorage:** Se aplicó como motor de persistencia temporal en el navegador web para gestionar el estado del carrito de compras y simular el manejo de la sesión de usuario y roles, cumpliendo estrictamente con los fines educativos requeridos para la Fase 1.

1.2. Tecnologías de Backend

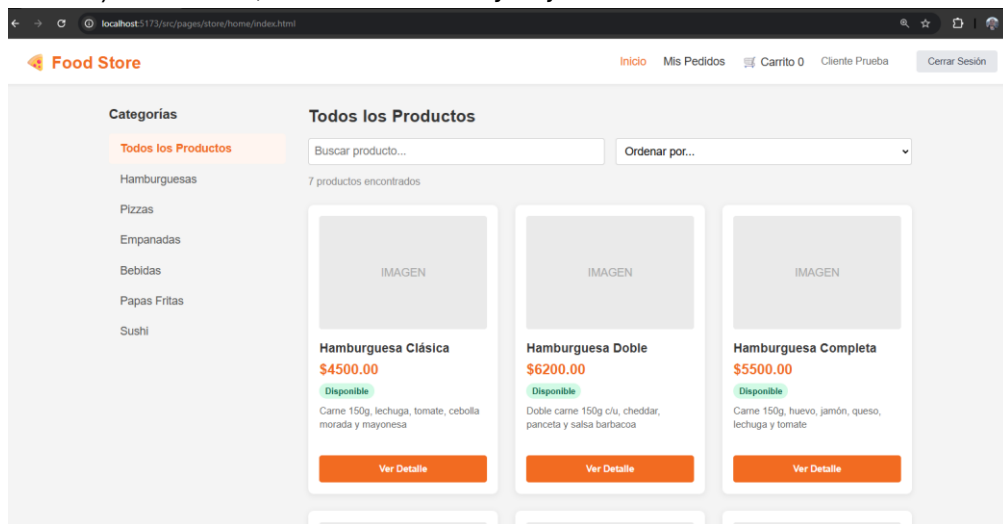
- **Java 23:** Lenguaje base para la construcción de la lógica de negocio y la interfaz de interacción mediante un menú por consola.
- **JPA e Hibernate:** Se adoptó la especificación Java Persistence API (JPA) junto con Hibernate como proveedor ORM (Object-Relational Mapping). Esta capa permitió abstraer las operaciones de base de datos, gestionando el ciclo de vida de las entidades, las transacciones atómicas y las relaciones (composiciones y agregaciones), así como la ejecución de consultas personalizadas mediante JPQL.
- **H2 Database Engine:** Se integró este motor de base de datos relacional embebido, configurado en modo archivo local (jdbc:h2:file:./data/jpa_db). Esta decisión arquitectónica garantizó la persistencia real de los datos del catálogo y pedidos asegurando la portabilidad del proyecto sin requerir la instalación de motores de bases de datos externos.

2. Decisiones Técnicas y Arquitectura

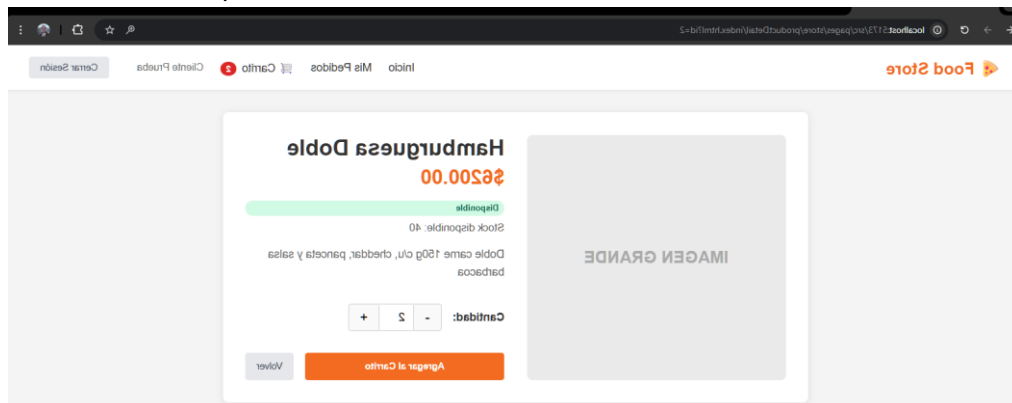
La solución arquitectónica se diseñó bajo una perspectiva minimalista y con una visión integral de 360 grados del proyecto. Se priorizó la mantenibilidad, la escalabilidad temprana y una separación clara de responsabilidades operativas entre el cliente web y el motor de gestión por consola.

Estrategia Frontend (Cliente Web)

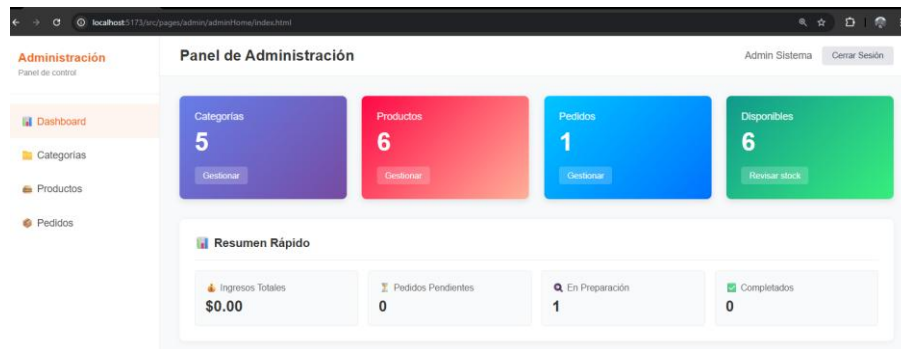
- **Hidratación Híbrida como Fuente de Verdad:** Ante la ausencia de una API REST de conexión directa en esta primera fase, se diseñó un modelo de hidratación que prioriza el almacenamiento local (localStorage). Los archivos estáticos .json operan exclusivamente como semilla de lectura inicial. Esta decisión asegura que las mutaciones transaccionales ejecutadas por el administrador (como el alta de categorías o la modificación del stock) se sincronicen y persistan en la vista del cliente.
- **Seguridad y Control de Rutas (Route Guards):** Se implementó una validación estricta de sesión en la cabecera del ciclo de vida de los scripts. Las interfaces protegidas inician ocultas mediante CSS (display: none;) y el DOM solo se renderiza si el script valida que el token de usuario posee el rol ADMIN, mitigando la exposición de contenido no autorizado.
- **Arquitectura de Interfaz:** Se optó por una manipulación directa del DOM (Vanilla TypeScript) para garantizar un empaquetado ligero, prescindiendo de frameworks pesados que agregarían complejidad innecesaria a los requisitos del flujo.
- **Captura del Catálogo (Cliente):** Vista principal del catálogo de productos (Módulo Cliente) con listado, sistema de filtros y tarjetas de información.



2. Captura del Carrito de Compras (Cliente): Figura 2: Interfaz del carrito de compras con control dinámico de cantidades, validación de stock y panel de confirmación de pedido.



- Captura del Dashboard (Admin): Figura 3: Dashboard principal del módulo administrativo exhibiendo métricas clave y estado general del negocio calculados en tiempo real.



Estrategia Backend (Consola Java)

- **Diseño por Capas y Patrón Repository:** El código se estructuró dividiendo claramente los modelos (entidades JPA) de la lógica de acceso a datos. Se implementó una clase abstracta `BaseRepository<T>` que concentra el comportamiento CRUD genérico. Esto garantizó el cumplimiento del principio DRY (Don't Repeat Yourself), aislando en los repositorios específicos (`ProductoRepository`, `PedidoRepository`) únicamente las consultas JPQL particulares.
- **Gestión Transaccional Atómica:** El ciclo de vida de las conexiones a la base de datos se centralizó a través de un patrón Singleton en la clase `JPAUtil`. Para procesos críticos de concurrencia, como el "Alta de Pedido", se diseñaron transacciones atómicas de un solo hilo (`EntityManager` único) que aseguran el cálculo de subtotales, la cascada de los `DetallePedido` y el descuento del inventario en un único bloque de confirmación (`commit`), ejecutando un `rollback` inmediato ante cualquier inconsistencia.

The screenshot shows a Java IDE with a project explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays a `Main.java` file with imports for `com.tp.jpa.repository.UsuarioRepository`, `com.tp.jpa.util.JPAUtil`, and `jakarta.persistence.EntityManager`. The terminal shows the output of a Java application, including a menu and a successful category creation message.

```

14 import com.tp.jpa.repository.UsuarioRepository;
15 import com.tp.jpa.util.JPAUtil;
16 import jakarta.persistence.EntityManager;
17
18 import java.util.Scanner;

SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
PS C:\TP Integrador> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '@C:\Users\hugo\AppData\Local\Temp\cp_3174912\
jz10aa3j2f2tr27cz3.argsfile' 'com.tp.jpa.Main'
0m:nInitiator{ConnectionProvider3dbcConnectionAccess$509ac12c} for (non-JTA) DDL execution was not in auto-commi
t mode; the Connection 'local transaction' will be committed and the Connection will be set into auto-commit mode.

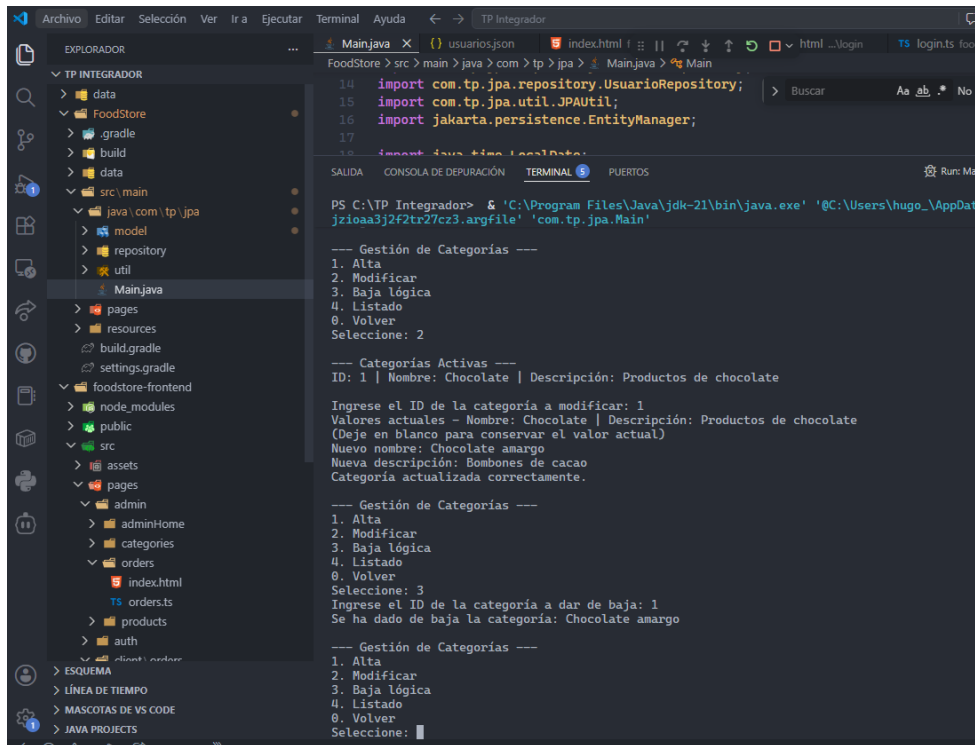
=== MENÚ PRINCIPAL ===
1. Gestionar Categorías
2. Gestionar Productos
3. Gestionar Usuarios
4. Gestionar Pedidos
5. Reportes
0. Salir
Seleccione una opción: 1

--- Gestión de Categorías ---
1. Alta
2. Modificar
3. Baja lógica
4. Listado
0. Volver
Seleccione: 1
Nombre de la categoría: Chocolate
Descripción: Productos de chocolate
Categoría creada con éxito. ID asignado: 1

--- Gestión de Categorías ---
1. Alta
2. Modificar
3. Baja lógica
4. Listado
0. Volver
Seleccione: 2

--- Categorías Activas ---
ID: 1 | Nombre: Chocolate | Descripción: Productos de chocolate
Ingrese el ID de la categoría a modificar:

```



3. Dificultades y Soluciones (Análisis de Deuda Técnica)

Durante el desarrollo de los Sprints correspondientes a la Fase 1, se enfrentaron desafíos arquitectónicos y de sincronización de estado, los cuales fueron resueltos mediante implementaciones de control local. A continuación, se detalla el análisis de la resolución y la deuda técnica adquirida que requerirá refactorización en etapas de integración posteriores:

- **Flash of Unauthorized Content (FOUC):** Al implementar la protección de rutas para el panel de administración, el ciclo de vida diferido de los módulos ES en Vite provocaba que el árbol DOM se renderizara fracciones de segundo antes de que el script validara los permisos, exponiendo visualmente la estructura del dashboard a usuarios no autorizados.
- **Solución FOUC:** Se invirtió la lógica de visibilidad estructural. El documento HTML se inicializa bloqueado nativamente mediante la propiedad CSS display: none en la etiqueta base. El controlador de TypeScript asume la responsabilidad exclusiva de mutar el estado a visible únicamente tras verificar de forma estricta la existencia del rol ADMIN en la sesión.
- **Sincronización Híbrida de Estado:** En la iteración inicial, los módulos del cliente consumían datos de lectura fijos desde los archivos .json, mientras que el módulo de administración mutaba el inventario persistiendo en el localStorage. Esto generó un desacople de la información donde el cliente no reflejaba los cambios de disponibilidad en tiempo real.
- **Solución de Sincronización:** Se estableció un origen único de datos (Single Source of Truth). Se refactorizó la lógica asíncrona en todas las vistas para que consuman prioritariamente la memoria dinámica del navegador. Los archivos

.json quedaron relegados a operar exclusivamente como semilla de hidratación inicial si el almacenamiento local se encuentra vacío.

- **Deuda Técnica - Vulnerabilidad de Autenticación:** El sistema actual delega el control de sesión y roles al cliente mediante almacenamiento local, lo cual expone la aplicación a manipulación de estado. Se requerirá la implementación de seguridad robusta mediante validación por tokens (JWT) emitidos por el backend.
- **Deuda Técnica - Concurrencia Transaccional:** La gestión del carrito en memoria del navegador carece de bloqueos optimistas o pesimistas contra la base de datos real. Al no existir control de concurrencia global, el sistema permite teóricamente la sobreventa si múltiples clientes operan sobre el mismo artículo de forma simultánea.

4. Enlaces de Entrega

- **Repositorio de Código (GitHub):** [Insertar enlace al repositorio aquí]
- **Video de Demostración:** [Insertar enlace público del video aquí]

5. Bibliografía / Webgrafía

- **TypeScript Documentation:** Referencia de tipado estático y manipulación del DOM nativo. Disponible en: <https://www.typescriptlang.org/docs/>
- **Vite:** Herramienta de empaquetado modular y entorno de desarrollo. Disponible en: <https://vitejs.dev/guide/>
- **MDN Web Docs:** Documentación técnica sobre HTML5, CSS3, ES Modules y Web Storage API (localStorage). Disponible en: <https://developer.mozilla.org/>
- **Hibernate ORM / JPA:** Especificación de mapeo objeto-relacional y gestión transaccional. Disponible en: <https://hibernate.org/orm/documentation/>
- **H2 Database Engine:** Documentación del motor de base de datos relacional embebido. Disponible en: <https://h2database.com/html/main.html>

